

Rethinking the 2,000-Location Model

By: Sai Rashwant Venkataraman Sundaram
Abrightlab Technology Associate, Software Engineering

Live prototype: https://sairashwant.github.io/abrightlab_project/

Code: https://github.com/sairashwant/abrightlab_project

Summary. Losing locations are not one problem, so negotiating vendor prices across the board will not fix them. Each location loses money for a specific, measurable reason that Peazy's data already reveals, and each reason has a different fix. I built a working prototype that diagnoses all 2,000 locations, redesigns vendor territories and routes, and tests a recovery plan before it touches a customer. On synthetic data, the plan cuts losing locations from 612 to 138 and lifts gross margin from 5.5% to 17.4%.

1. Problem framing and assumptions

I treat each location as its own monthly P&L: contract revenue minus vendor cost (labor, travel and rework). A location loses money when cost exceeds revenue, and underperforms when its margin falls below the 25% target its price assumed. Key assumptions, all editable in one config file: a 25% target margin, 3,000 sq ft cleaned per hour, about \$20 per driving hour for vehicles, and territories of about 10 sites. Without access to Peazy data, the prototype generates a realistic synthetic network of 2,000 locations across 15 metros and 65 vendors, with 12% remote sites. I planted known problems (underpriced contracts, scope creep and over-billing vendors) so the diagnosis can be checked against right answers.

2. Additional data needed and why

Data	Why it is needed
Geofence check-in and check-out per visit, and vendor invoice lines	Compare on-site hours with billed hours to find time paid for but not worked.
Task checklists, completed versus contracted	Find unpaid scope creep, and give evidence for change orders.
Contract price, square footage, frequency, renewal dates	Find underpriced contracts and know when each can be repriced.
Location and vendor base coordinates, plus road drive times	Measure travel cost and density. Drive times come from OpenStreetMap (OSRM).
Inspection scores, issues, callbacks, customer renewals	Guardrails, so savings never come from cutting quality or losing customers.

Most of this already exists in Peazy. The main new work is joining vendor invoices to individual visits.

3. How I would identify root causes

For each location, the gap between its target margin and its actual result is split into four dollar amounts, each from one signal: **underpricing** (what the contract should cost minus what the customer pays), **travel** (drive cost above what the price assumed), **scope creep** (work beyond the contracted scope, from task checklists) and **billing gap** (billed hours minus geofence hours). The largest amount is the location's primary cause. This is plain accounting, not a black-box model, so every dollar traces back to a record operations can check.

Results: travel and low density is the largest cause (369 sites, \$6.2M a year), followed by underpricing (119 sites, \$1.0M), scope creep (84 sites, \$0.7M) and billing gaps (40 sites, \$0.2M). Checked against the planted problems, the diagnosis finds 86% of underpriced and 84% of scope-creep sites.

4. Solution and the role of technology

The biggest problem is structural: each metro's sites are split among several vendors, each driving from its own base. The plan combines five fixes:

- **Territory consolidation.** K-Means clusters each metro's sites into territories of about 10, and each territory goes to one of the metro's most accurately billing vendors. Nearest-neighbour plus 2-opt local search then builds each crew route from real OpenStreetMap drive times, the same approach I used to optimize Northeastern's RedEye cab routes. In New York, 7 vendors crisscrossing the city become 24 territory routes with 63% less driving.
- **Local vendors for remote sites** instead of metro crews driving 30 to 100 miles.
- **Pay for verified hours:** invoices are reconciled automatically against geofence time.
- **Change orders for added scope,** backed by Peazy's task checklists.
- **Should-cost repricing at renewal,** capped to protect customer relationships.

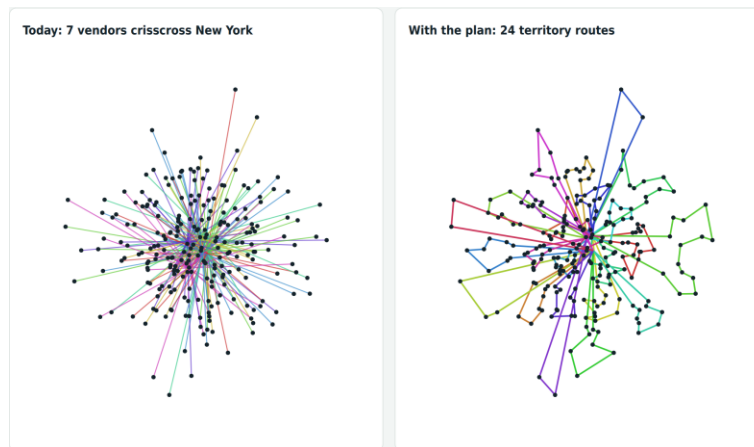


Figure 1. New York today (left) and with territory routes (right).

A plan simulator estimates each fix per location and tests combinations. With conservative defaults, such as closing half of each price gap and consolidating only 30% of territories, the plan adds \$14.2M a year in gross margin. The stack is Python (scikit-learn for clustering), the OSRM routing API, a Flask server and a browser dashboard. In production, the engine would read Peazy's database or API in place of the synthetic generator.

5. How I would measure success

Primary metric: the share of locations losing money. **Supporting metrics:** gross margin per location and per square foot, the gap between billed and geofence hours (target under 3%), and drive time per visit. **Guardrails:** inspection scores, callbacks, customer renewals and vendor retention, so savings never come from cutting quality. Every pilot result is compared against a similar control metro.

6. What I would build or test first

1. **Location P&L and diagnosis on real data (weeks 1 to 3).** It changes nothing for customers or vendors, so it is low risk, and it shows where the money is.
2. **Verified-hours check (weeks 3 to 6).** A review queue that flags invoices more than 10% above geofence time.
3. **90-day territory pilot** in one highly fragmented metro, measured against a control metro.

Limitations and AI use

The synthetic data proves the method, not the dollars, so the first step on real data is recalibrating every assumption. Routing treats each territory as one loop; daily dispatch would need time windows and crew capacity, using a solver such as Google OR-Tools. Fix estimates are made independently, so small overlaps are not netted out.

I used Claude (Anthropic) throughout: to brainstorm the approach, write and simplify the prototype code, and draft this write-up. I set the direction and design choices, including minimal code, keeping all inputs in a config file, using OpenStreetMap drive times and adding a one-click rebuild. I ran and tested the prototype on my own machine with live OpenStreetMap data, and edited this document.